LSV Analysis: iR Compensation, Tafel Slope Analysis

A **Streamlit GUI** with two independent analytical workflows for electrochemical polarization data:

1. **iR compensation** – ohmic-drop correction of linear-sweep voltammetry (LSV) data, using the uncompensated resistance **Ru** estimated from electrochemical impedance spectroscopy (EIS).
2. **Tafel slope analysis** – fits the linear (activation-controlled) region of a polarization curve to extract the Tafel slope, with RHE reference conversion and HER/OER/ORR mechanistic interpretation.

■ **Live app:** https://lsv-analysis-ir-compensation-tafel-slope.streamlit.app/

> Estimate `Ru` from an EIS Nyquist arc, reconcile current/resistance units (with electrode area when needed), then apply a user-selected iR-compensation factor (5 – 100 %; 85 % recommended) to your LSV curve and export the corrected data — or, independently, upload one or more polarization curves and extract a publication-ready Tafel slope per sample.

Workflow

```
 Excel / CSV ■■  EIS (Z', Z'')  ■■  circle fit  ■■  Ru  ■
                                                    ■■■  E_corr = E – (f%)·I·Ru  ■■  CSV
               LSV (Potential, Current)  ■■■■■■■■■■■■■■■■■

Excel / CSV (one or more files) ■■ RHE conversion ■■ onset-aware
  linear-region fit ■■ E = a + b·log|i|  ■■  Tafel slope (mV/dec)
```

1 · EIS / Ru Analysis + LSV iR Correction tabs

1. **Load data** – an Excel workbook where **Sheet 1 = EIS** (`Z'`, `Z''`) and **Sheet 2 = LSV** (`Potential`, `Current`), or two separate CSV files. Each sheet may hold **several datasets side-by-side as repeated column pairs** (`Z'_1, Z''_1, Z'_2, Z''_2, …` and `E_1, I_1, E_2, I_2, …`). The app parses every pair and **links EIS sample *i* to LSV sample *i***; pick the active sample from the sidebar.
2. **Units & electrode area** – tell the app how the LSV current and the EIS resistance are reported so that `I·Ru` resolves to **volts**. Units are **auto-detected from the column headers** (e.g. `Current (mA/cm2)`, `Z' (Ohm)`) and shown in the sidebar: - *Absolute current* (mA, A, …) pairs with **Ru in Ω**. - *Current density* (mA/cm², …) pairs with **Ru in Ω·cm²**. - When one side is per-area and the other is not, an **electrode area (cm²)** reconciles them (`Ru[Ω·cm²] = Ru[Ω] · area`); when the units are already consistent the area input is bypassed.
3. **EIS / Ru Analysis tab** – the kinetic semicircle of the Nyquist plot is fitted with an algebraic circle fit. Its **high-frequency real-axis intercept is Ru**; the low-frequency intercept gives `Ru + Rct`. The low-frequency diffusion tail is auto-excluded and the fit range is fully adjustable. You can also enter Ru manually. When the current is a density, the impedance is reported **area-normalised in Ω·cm²** (with the original raw Ω value shown alongside). With multiple samples loaded, a **batch table** lists the fitted `Ru` for every sample.

4. **LSV iR Correction tab** – pick one *or several* compensation factors (5 – 100 %; 85 % recommended). The app computes

```
text E_corrected = E_measured – (factor / 100) · I · Ru
```

(with current scaled to its base SI unit and Ru reconciled via the area, so the drop is in volts). It shows a **with-vs-without comparison** (overlay or side-by-side), a results table with **Ru (Ω)**, **Ru effective (Ω·cm²)**, and the max iR drop, and lets you **download a CSV** that embeds `Ru`, the compensation %, and one column block per factor. 5. **Over-compensation guard** – each factor is checked for *fold-back* (the corrected potential reversing instead of advancing). Over-corrected factors are flagged with a ■ status, and the app reports the **highest safe factor** for that sample plus an interpretation of what a good compensation looks like.

Why Ru comes from a circle fit (not an equivalent-circuit fit)

The reference dataset stores only `Z'` and `Z''` with **no frequency column**, so a frequency-dependent equivalent-circuit (e.g. Randles) fit is not possible. The geometric circle fit recovers Ru directly from the Nyquist arc and needs no frequency data.

2 · Tafel Slope Analysis tab

Fully independent of the tabs above — its own file uploader, own units, and its own reference-electrode handling.

- **Data source.** Upload one or more Excel workbooks or CSV files (one per sample/lot); each file (or repeated column pair within a file) becomes a sample you can select into a single **journal-style overlay plot**.
- **Reference electrode → RHE conversion.** Electrochemical Tafel/HER/OER/ORR data is conventionally reported vs. the reversible hydrogen electrode (RHE). If your data isn't already on that scale, pick the reference electrode used (SHE, SCE, Ag/AgCl variants, Hg/HgO, Hg/Hg■SO■, or a custom E° vs NHE) and the electrolyte pH; the app converts via `E(RHE) = E(measured) + E°(ref vs NHE) + 0.0592 V·pH■¹ × pH`.
- **Current unit & electrode area.** Optionally convert an absolute current (e.g. mA) to a current density (mA/cm²) by dividing by an electrode area, the same way the iR-correction tabs do.
- **Onset-aware auto fit range.** For each sample, the linear (kinetic) Tafel region is auto-detected starting close to the reaction **onset** — where `|current|` first departs from the flat background/capacitive baseline — and grown outward while `E` vs `log|i|` stays linear ($R^2 \geq 0.99$), stopping once real curvature (e.g. the mass-transport-limited plateau) sets in. Each sample's range is independently adjustable via a **potential-labeled slider**, or by **box-selecting the region directly on the Tafel plot**.
- **Per-sample reaction type.** Each sample is tagged HER, OER, ORR, or Other/unspecified; samples of different reaction types can be combined in one overlay, with the **legend split into per-reaction groups**.
- **Results.** Tafel slope (mV/dec, always reported as its positive magnitude per literature convention), R^2, and — for HER specifically, since 0 V vs RHE is exactly the H■/H■ equilibrium potential — the extrapolated exchange current i■. Each sample's fitted slope is compared against canonical mechanistic benchmarks (e.g. ~120 mV/dec for a Volmer rate-determining step) and summarized in a short auto-generated analysis paragraph per reaction.
- **Journal-style export.** Both the Tafel plot and the Original LSV (linear, pre-log) polarization curve are rendered with an outer box border, no interior gridlines, Arial fonts at a selectable 28/36 pt, and are downloadable as PNGs (plot, and a separately rendered results table), along with a CSV of the summary

table.

Installation

Requires Python 3.10+.

```
pip install -r requirements.txt
```

Dependencies: `numpy`, `scipy`, `pandas`, `openpyxl`, `streamlit`, `plotly`, `kaleido` (server-side PNG export).

Running the app

```
streamlit run app.py
```

Then open the URL shown in the terminal (default `http://localhost:8501`). Select **Use bundled sample** in the sidebar to try the EIS/LSV tabs immediately with `sample-data/Book1-original data.xlsx` (current in mA/cm², EIS in Ω; electrode area 0.04 cm²). The Tafel Slope Analysis tab has its own independent uploader and works with any Potential/Current polarization data.

Deploy publicly on Streamlit Community Cloud

This app is ready to deploy for free on Streamlit Community Cloud:

1. Push this repository to GitHub (it already contains `app.py` and `requirements.txt` at the root, which is all the platform needs).
2. Go to **share.streamlit.io** → **Create app**, pick this repo/branch, and set the **main file path** to `app.py`.
3. Click **Deploy**. The platform installs `requirements.txt` and serves the app at `https://<your-app-name>.streamlit.app`.

The sample workbook ships in `sample-data/`, so the deployed app works out of the box; users can also upload their own Excel/CSV files.

Keeping the source code private

Streamlit Community Cloud runs the app **directly from the GitHub repo it is linked to**, so the code must live in that repo — you cannot `.gitignore app.py` and still deploy it (the platform would not have the file). To publish a working app while keeping the source hidden from the public, **make the GitHub repository private**: Community Cloud can deploy from private repos (authorize repo access when creating the app). The deployed app stays publicly reachable at its `*.streamlit.app` URL, while the code is not visible to anyone browsing GitHub. (You can additionally restrict app *viewers* in the app's **Settings** → **Sharing** if you also want to limit who can use it.)

Project layout

Path	Purpose
`app.py`	Streamlit GUI (EIS/Ru analysis, LSV iR-correction, and Tafel slope analysis tabs).
`ir_compensation/data_io.py`	Load EIS / LSV / polarization data from Excel or CSV (fuzzy column matching).
`ir_compensation/eis.py`	Circle fit of the Nyquist arc → `Ru`, `Rct`.
`ir_compensation/correction.py`	Apply the iR correction (5–100 % factor; current/Ru unit & area reconciliation).
`ir_compensation/tafel.py`	Onset-aware Tafel-region fit, RHE reference-electrode conversion, mechanistic benchmarks.
`sample-data/Book1-original data.xlsx`	Example data: `EIS` sheet (Z′, Z″ in Ω), `LSV` sheet (Potential V, Current mA/cm²).

Using the core API without the GUI

```
from ir_compensation import data_io, eis, correction, tafel

eis_d = data_io.load_eis("sample-data/Book1-original data.xlsx", sheet=0)
lsv_d = data_io.load_lsv("sample-data/Book1-original data.xlsx", sheet=1)

ru = eis.fit_ru_circle(eis_d.z_real, eis_d.z_imag).ru     # ≈ 27.5 Ω

# Current density (mA/cm²) + Ru in Ω → reconcile with the electrode area.
result = correction.apply_ir_correction(
    lsv_d.potential, lsv_d.current, ru,
    factor_percent=85, current_unit="mA/cm²",
    ru_unit="Ω", area_cm2=0.04,
)
result.potential_corrected   # iR-corrected potentials (V)
result.ru_effective          # Ru reconciled to Ω·cm² (= ru · area)

# Absolute current (mA) + Ru in Ω needs no area:
correction.apply_ir_correction(
    lsv_d.potential, lsv_d.current, ru,
    factor_percent=100, current_unit="mA",  # 100 % = full correction
)

# Tafel slope from a polarization curve, converting Ag/AgCl (sat. KCl) at
# pH 13 to the RHE scale first.
pot_rhe = tafel.to_rhe(lsv_d.potential,
                       tafel.REFERENCE_ELECTRODES["Ag/AgCl, saturated KCl"],
                       ph=13.0)
log_i = tafel.log_current(lsv_d.current)
start, stop = tafel.auto_tafel_range(pot_rhe, log_i, current=lsv_d.current)
fit = tafel.fit_tafel(pot_rhe, log_i, start, stop)
fit.slope_mv_per_dec         # Tafel slope, mV/decade
fit.exchange_current         # extrapolated current at E(RHE) = 0
```

Notes

- **Unit consistency.** Absolute-current units (`A`, `mA`, `µA`, `nA`) pair with **Ru in Ω**; current-density units (`A/cm²`, `mA/cm²`, …) pair with **Ru in Ω·cm²**. The current is scaled to its base SI unit (e.g. mA → A divides by 1000) and, when only one side is area-normalised, the **electrode area** converts Ω ↔ Ω·cm² — so `I·Ru` always resolves to volts. Units are guessed from the column headers and can be overridden in the sidebar.
- **Compensation factor** is selectable from **5 % to 100 %**. 100 % is the full ohmic-drop correction; **85 % is the recommended safe default**, since full positive feedback can over-correct/oscillate when `Ru` is uncertain. The fold-back guard flags any factor that over-corrects.
- **Tafel slope sign.** The fitted slope is always reported as its **positive magnitude** (mV/dec), matching literature convention, regardless of whether the underlying reaction is anodic (OER) or cathodic (HER/ORR).

Security

Because the app accepts arbitrary uploaded files on a public URL, several safeguards are built in:

- **Upload limits.** `.streamlit/config.toml` caps upload size (15 MB) and message size, keeps XSRF protection on, disables usage telemetry, and hides Python tracebacks from end users (`showErrorDetails = false`).
- **Malicious-file defenses** (`ir_compensation/data_io.py`): Excel archives are checked for **decompression-bomb** expansion before parsing; tables are rejected above row/column caps (memory-exhaustion / DoS guard); the number of parsed column-pairs is capped; and header-derived labels are stripped of control characters / newlines before display or CSV output (prevents CSV/markdown injection).
- **No persisted data.** Uploads are processed in memory only; nothing is written to disk. The sample is **not preloaded**, and a **Clear / reset** button wipes session data.
- **Optional password gate.** Set an `app_password` secret (see `.streamlit/secrets.toml.example`) to require a password; comparison is constant-time. Leave it unset for a fully public app. `secrets.toml` is git-ignored, so credentials are never committed.

To restrict *who* can open the app entirely, you can also limit viewers under the app's **Settings** → **Sharing** on Streamlit Community Cloud.

Citation

If you use this app (including a public Streamlit deployment) in your work, please cite it. Machine-readable metadata is in `CITATION.cff`; a plain-text form:

> Kumar, R. (2026). *LSV Analysis: iR Compensation and Tafel Slope Analysis* (v1.1.0) [Computer software]. North Carolina Central University. https://github.com/rajeev4187/LSV-Analysis-iR-compensation-Tafel-slope

```
@software{kumar_lsv_analysis_2026,
 author  = {Kumar, Rajeev},
 title   = {LSV Analysis: iR Compensation and Tafel Slope Analysis},
 version = {1.1.0},
 year    = {2026},
 url      = {https://github.com/rajeev4187/LSV-Analysis-iR-compensation-Tafel-slope}
}
```

License

Released under the MIT License — see `LICENSE`.